

1. What is a Microservice?

- Small, independently deployable service focused on a single business capability.
 - Runs in its own process and communicates via REST, gRPC, or messaging.
 - Owns its database to maintain loose coupling.
 - Can be developed, deployed, and scaled independently.
-

2. Advantages over Monolith

- Independent deployment of services.
 - Better scalability (scale only required service).
 - Fault isolation improves system stability.
 - Technology flexibility per service.
-

3. Disadvantages

- Distributed system complexity.
 - Network latency and partial failures.
 - Harder debugging and monitoring.
 - Operational overhead (DevOps, containers, CI/CD).
-

4. When NOT to use Microservices

- Small team or simple application.
 - Early startup phase.
 - Lack of DevOps/cloud maturity.
 - When domain is not clearly bounded.
-

5. Design Principles

- Single Responsibility per service.
 - Bounded Context (DDD).
 - Database per service.
 - API-first design and automation.
-

6. 12-Factor Methodology

- Config externalized from code.
 - Stateless processes.
 - Dev/Prod parity.
 - Logs treated as event streams.
-

7. Why Stateless?

- Improves horizontal scalability.
 - Any instance can serve requests.
 - Session stored externally (Redis/DB).
 - Better fault tolerance.
-

8. Why Spring Boot + Spring Cloud?

- Spring Boot simplifies setup and auto-configuration.
 - Spring Cloud provides service discovery.
 - Centralized config management.
 - Resilience patterns support.
-

9. Shared Database?

- Not recommended.
 - Causes tight coupling.
 - Breaks independent deployment.
 - Violates bounded context.
-

10. Distributed Tracing

- Tracks request across services.
 - Helps identify latency bottlenecks.
 - Improves debugging.
 - Uses trace and span IDs.
-

11. Tracing Implementation

- Add correlation ID in headers.
 - Use Spring Cloud Sleuth.
 - Integrate with Zipkin.
 - Monitor request flow visually.
-

12. Communication Methods

- Synchronous REST/gRPC.
 - Asynchronous messaging (Kafka/RabbitMQ).
 - External via API Gateway.
 - Event-driven architecture.
-

13. Design Patterns Used

- API Gateway.
 - Circuit Breaker.
 - SAGA pattern.
 - CQRS.
-

14. Database Patterns

- Database per service.
 - Event sourcing.
 - CQRS read/write separation.
 - Shared DB (anti-pattern).
-

15. SAGA Pattern

- Handles distributed transactions.
 - Series of local transactions.
 - Compensating actions on failure.
 - Choreography or Orchestration model.
-

16. CQRS

- Separate read and write models.
 - Optimizes performance.
 - Improves scalability.
 - Common in event-driven systems.
-

17. Read vs Write Heavy

- Read-heavy: caching + CQRS.
 - Write-heavy: event sourcing.
 - Async processing.
 - Optimized write database.
-

18. Choreography vs Orchestration

- Choreography: event-driven, no central control.
 - Orchestration: central coordinator.
 - Choreography loosely coupled.
 - Orchestration easier to manage.
-

19. Fault Tolerance

- Circuit Breaker.
 - Retry mechanism.
 - Timeouts.
 - Bulkhead isolation.
-

20. Circuit Breaker Example

- Stops calling failing service.
 - Moves to open state after threshold.
 - Provides fallback response.
 - Prevents cascading failures.
-

21. Spring Annotations

- `@CircuitBreaker`.
 - `@Retry`.
 - `@RateLimiter`.
 - `@Bulkhead`.
-

22. Libraries

- Resilience4j.
 - Hystrix (deprecated).
 - Spring Retry.
 - Spring Cloud Circuit Breaker.
-

23. Async in Spring

- Use `@Async` annotation.
 - Return `CompletableFuture`.
 - Enable with `@EnableAsync`.
 - Improves non-blocking execution.
-

24. Async Microservice Call

- Kafka or RabbitMQ messaging.
 - `WebClient` (reactive).
 - Event-driven architecture.
 - Decouples services.
-

25. Communication Styles

- Synchronous HTTP.
 - Asynchronous messaging.
 - Event-driven.
 - Reactive streaming.
-

26. Restrict Service Calls

- OAuth2 and JWT.
 - API Gateway filtering.
 - Role-based access control.
 - Network security groups.
-

27. Secure Credentials

- Spring Cloud Config.
 - Vault integration.
 - Encrypted properties.
 - Environment variables.
-

28. Service Discovery

- Dynamic service registration.
 - Client-side load balancing.
 - Using Eureka or Consul.
 - Removes hardcoded URLs.
-

29. Config Management

- Central config server.
 - Environment-specific configs.
 - Dynamic refresh.
 - Encrypted properties.
-

30. API Versioning

- URI versioning (/v1).
 - Header versioning.
 - Backward compatibility.
 - Deprecation strategy.
-